

GCC target for educational RISC-V processor

Trabalho de Conclusão de Curso

Guilherme Stabach Salustiano

Orientador: Prof. Bruno de Carvalho Albertini

Departamento de Engenharia de Computação e Sistemas Digitais (PCS)
Escola Politécnica da Universidade de São Paulo (EPUSP)

2026

Sumário

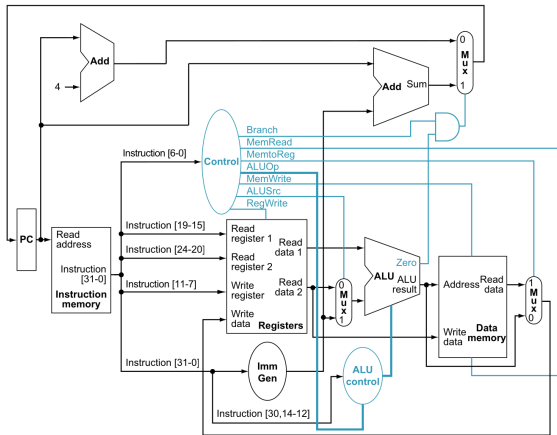
Roteiro da apresentação

- 1 Contexto e Motivação
- 2 Trabalhos Relacionados
- 3 Desenvolvimento
- 4 Validação
- 5 Resultados
- 6 Conclusão
- 7 Trabalhos Futuros

O processador Hennessy–Patterson

Contexto

- *Computer Organization and Design: RISC-V Edition* é a referência em curso de arquitetura de computadores.
- Usado em Sistemas Digitais II (PCS3225) na Poli.
- Implementa um processador mono-ciclo com 8 instruções: lw sw beq add addi sub and or



Fonte: Patterson & Hennessy, *Computer Organization and Design: RISC-V Edition*, 2ª ed., 2020, Figura 4.21.

Não há compilador para apenas 8 instruções.

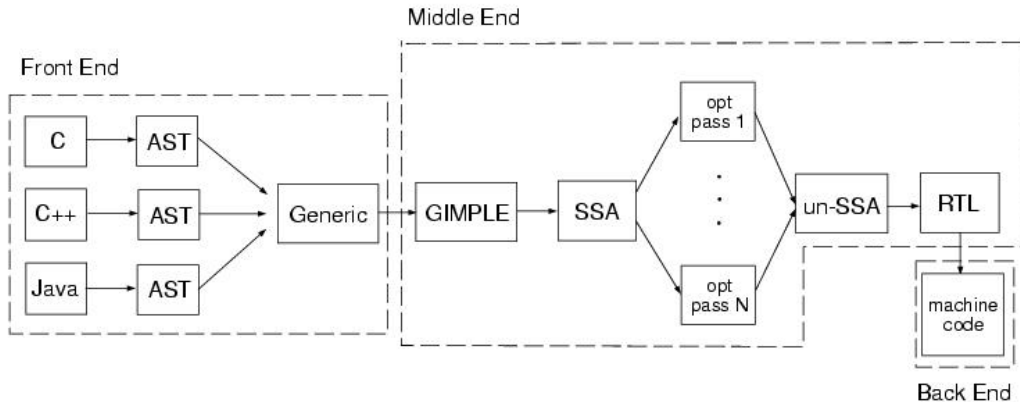
- GCC e Clang geram no mínimo 40 instruções (RV32I).
- Alunos e professores precisam escrever os programas manualmente em assembly, limitando o tamanho dos programas.
- Perde-se a oportunidade de integrar o conhecimento de C do aluno, introduzindo a tradução e otimização de compiladores.

Gerar assembly a partir de C para conjuntos de instruções progressivos, sintetizando toda instrução ausente com uma sequência equivalente no GCC.

Alvo	ISA
rvsc0	lw sw beq add addi sub and or
rvsc1	+ lui, jalr
rvsc2	RV32I – fence – Zicsr
rvsc3	RV32I completo
rvsc4	RV64I
rvsc5	RV64IM
rvsc6	RV64IMFD + Zicsr
rvsc7	RV64IMAFD

- **Berkeley SoftFloat** — ponto flutuante em software.
- **MSP430**: `__mspabi_slli_n` — shift de vários bits em ISA single shift.
- **AVR**: `__mulhi3` — multiplicação em software.

GCC Pipeline



Fonte: AlexeySmirnov, *GNU C Compiler Internals* (Wikibooks), arquivo Gcc.JPG, CC BY-SA 3.0.

Exemplo: XOR

Codigo

```
int f(int a, int b) { return a ^ b; }
```

GIMPLE

```
D.2418 = a ^ b;  
return D.2418;
```

RTL

```
(set (reg:SI 137) (xor:SI (reg:SI 135) (reg:SI 136)))
```

Assembly

```
f:  
    xor a0, a0, a1  
    ret
```

Exemplo: "soft" XOR

RTL

```
(set (reg 138) (and:SI (reg 135) (reg 136)))  
(set (reg 139) (ior:SI (reg 135) (reg 136)))  
(set (reg 137) (minus:SI (reg 139) (reg 138)))  
  (REG_EQUAL (xor:SI ...))
```

Assembly

```
f:  
    and a5, a0, a1  
    or a0, a0, a1  
    sub a0, a0, a5  
    ret
```

Exemplo: SLL

$$\text{sll}(x, 0) = x$$

$$\text{sll}(x, n+1) = \text{sll}(x, n) + \text{sll}(x, n)$$

- Se n imediato:

```
add rd, rd, rd  
... # n vezes
```

- Se n parametrizado:

```
for (uint32_t rd = rs1, i = 0; i < rs2; i++) rd += rd;
```

Operação	Pior caso	Regs extras
not	2	0
xor (reg / imm)	3 / 4	1 / 2
ori / andi	2	1
sll const / var	31 / 47	0 / 1
srl const / var	158 / 332	3
sra const / var	194 / 391	5 / 7
slt / sltu	49 / 48	3 / 4
bne	3	1
blt,bltu / bge,bgeu	16 / 17	7
lb/lbu	77 / 460	2
lh/lhu	85 / 532	2
sb	32 / 95	3
sh	23 / 82	3
lui (rvsc0)	25	0
jal (rvsc1)	5	1

- **rvsc0**

- 12 *smoke tests* escritos à mão.

- **rvsc1**

- 8 *smoke tests* escritos à mão.
- GCC *torture suite*: 1684 programas usados pelo próprio GCC.

Cada teste em 5 níveis de otimização (-O0 ... -Os).

- **Validação estática:** `objdump -M no-aliases` e checagem das operações geradas contra a lista permitida.
- **Validação dinâmica:** programas validados por `assert` no próprio código e executados no emulador **Spike**.

Torture suite test (rvsc1)

Resultado

- 13 testes precisaram ser pulados:
 - `run_expensive_tests` (timeout)
 - `c99_runtime`
 - `int128`
 - `mmap` e `dfp`
- **1669 testes passaram** gerando apenas as 10 instruções do `sc1`, totalizando **8345 execuções**.

Embench-IoT benchmark

Resultado

- Suíte livre de benchmarks para **IoT / embarcado** — programas reais e pequenos, sem dependência de sistema operacional.
- **19 casos**: criptografia (`nettle-aes`, `nettle-sha256`, `md5sum`, `crc32`), processamento de sinal e imagem (`edn`, `picojpeg`, `depthconv`, `qrduino`), estruturas de dados e strings (`sglib-combined`, `slre`, `wikisort`, `huffbench`, `tarfind`), aritmética e controle (`aha-mont64`, `matmult-int`, `ud`, `xgboost`, `statemate`, `nsichneu`).
- Duas medidas independentes: **tamanho** (`.text` a `-O2`) e **tempo** (instruções retiradas no Spike).
- Três *toolchains*: **gcc17** (mesmo *fork*, `riscv32-unknown-elf`) / **rvsc2** (nativo) / **rvsc1** (sintetizado).

Custo estático

Resultados

Benchmark	gcc17	rvsc1	razão	Benchmark	gcc17	rvsc1	razão
aha-mont64	3.548	28.468	$\times 8,02$	picojpeg	15.360	201.204	$\times 13,10$
crc32	380	1.752	$\times 4,61$	qrduino	12.852	286.212	$\times 22,27$
depthconv	608	5.820	$\times 9,57$	sglib-combined	10.824	34.596	$\times 3,20$
edn	3.244	59.344	$\times 18,29$	slre	4.256	44.408	$\times 10,43$
huffbench	2.140	15.936	$\times 7,45$	statemate	6.484	25.160	$\times 3,88$
matmult-int	936	1.500	$\times 1,60$	tarfind	528	4.772	$\times 9,04$
md5sum	1.016	3.140	$\times 3,09$	ud	1.436	1.964	$\times 1,37$
nettle-aes	4.444	117.652	$\times 26,47$	wikisort	7.744	38.812	$\times 5,01$
nettle-sha256	6.976	138.472	$\times 19,85$	xgboost	624	9.456	$\times 15,15$
nsichneu	19.668	48.548	$\times 2,47$				
média geométrica							$\times 6,99$

rvsc2 é idêntico ao gcc17 nos 19 benchmarks.

Custo dinâmico

Resultado

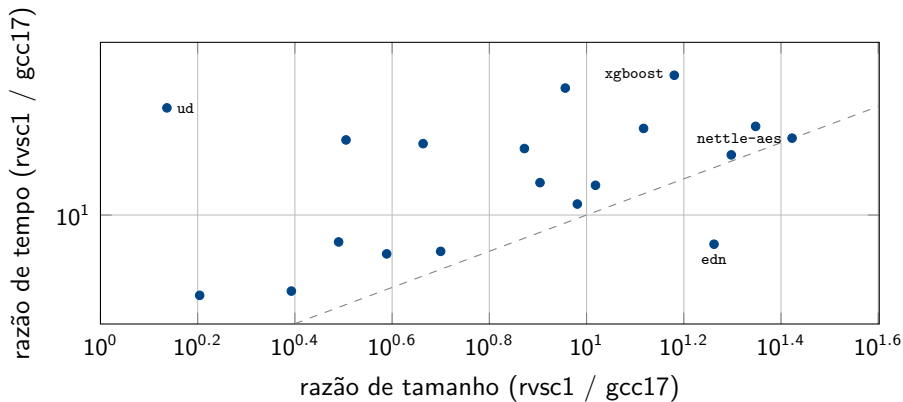
Instruções retiradas no Spike (milhões).

Benchmark	gcc17	rvsc1	razão	Benchmark	gcc17	rvsc1	razão
aha-mont64	13,0	196,6	× 15,1	picojpeg	3,6	108,4	× 30,1
crc32	6,0	148,5	× 24,8	qrduino	5,1	159,0	× 30,9
depthconv	54,6	626,3	× 11,5	sglib-combined	3,0	78,3	× 26,0
edn	68,6	476,3	× 6,9	slre	3,0	43,4	× 14,6
huffbench	2,4	55,9	× 23,3	statemate	2,1	12,6	× 6,1
matmult-int	24,4	86,8	× 3,6	tarfind	5,3	265,9	× 50,3
md5sum	2,9	20,7	× 7,1	ud	6,6	259,2	× 39,1
nettle-aes	4,7	124,4	× 26,6	wikisort	1,6	10,2	× 6,3
nettle-sha256	4,9	105,5	× 21,5	xgboost	3,8	225,3	× 59,2
nsichneu	2,5	9,5	× 3,8				
média geométrica							× 15,8

Todos os 19 benchmarks executam corretamente.

Custo estático x dinâmico

Resultado



Linha tracejada: tamanho = tempo. Acima dela, o custo é pago **dentro do laço**.

- Os objetivos foram cumpridos: `rvsc0` (oito instruções), `rvsc1` (convenção de chamada C completa com dez), `rvsc2–rvsc7` (RV32I $\rightarrow$ RV64IMAFD).
- Foram definidas derivações e otimizações para 31 instruções diferentes.
- Desenvolvimento de um framework de validação a partir de testes já existentes.
- **Custo:** $\times 6,99$ estático, $\times 15,8$ dinâmico. Sendo um valor aceitável para um curso introdutório.

- Otimizacao do SRL/SRA
- Distribuir binários pré-compilados, *startup files* e *linker scripts*
- Rodar o codigo gerado no processador de referencia da diciplina
- testar e medir os targets intermediários.

Referências

Fontes principais

- Patterson & Hennessy. *Computer Organization and Design: RISC-V Edition*, 2ª ed., Morgan Kaufmann/Elsevier, 2020 (Cap. 4.4; Figura 4.21).
- RISC-V ISA Specification; RISC-V psABI.
- GCC Internals Manual (*machine description*, `.opt`, `CC1_SPEC`).
- Spike — simulador de referência de ISA RISC-V.
- Embench-IoT — suíte de benchmarks embarcados.

Obrigado! Perguntas?